

Massively Parallel Conway’s Game of Life on AiMOS: Hybrid MPI + CUDA with Communication–Computation Overlap

Aidan LASTNAMEA

Rensselaer Polytechnic Institute
Troy, NY, USA
aidan@example.edu

Eric LASTNAMEB

Rensselaer Polytechnic Institute
Troy, NY, USA
eric@example.edu

Ben Werner

Rensselaer Polytechnic Institute
Troy, NY, USA
benwerner10@gmail.com

Abstract

Conway’s Game of Life is a deceptively simple cellular automaton whose naive implementation maps cleanly to a 2D stencil computation but whose communication structure becomes a first-class bottleneck once a realistic problem is distributed across thousands of GPUs. We present a hybrid MPI + CUDA implementation targeting the AiMOS DCS supercomputer at Rensselaer Polytechnic Institute, comprising 6 NVIDIA V100 GPUs per POWER9 node. Our system uses a 2D Cartesian rank topology with periodic boundary conditions, a two-phase east–west then north–south halo exchange that populates corner cells implicitly, three CUDA kernel variants (naive global-memory, shared-memory tiled, and an interior / boundary split for overlap), and CUDA-aware Spectrum MPI to hand device pointers directly to non-blocking communication primitives. We compare the CPU-only and hybrid CPU/GPU paths, measure strong and weak scaling on grids of up to $32,768^2$ cells, and quantify the speedup obtained by overlapping interior computation with halo exchange. The CPU-only build is $\sim 60\times$ slower than the hybrid build at matched rank counts, and the overlap-enabled hybrid build retains $\sim 82\%$ parallel efficiency at 48 GPUs across 8 nodes. We discuss the bandwidth and latency model that explains the observed scaling cliff, sketch a checkpoint-friendly MPI-IO subarray view, and outline directions for extending the system to HighLife and 3D continuous cellular automata.

1 Introduction

Conway’s Game of Life [?] is a two-state two-dimensional cellular automaton in which each cell at the next timestep is a fixed function of its eight Moore neighbors at the current timestep. The update rule (B3/S23) – a dead cell with exactly three living neighbors becomes alive, and a living cell with two or three living neighbors survives – yields a remarkable diversity of stable, periodic and propagating structures despite its triviality. Beyond its recreational appeal, the Game of Life is the canonical proxy for any nearest-neighbor stencil: the same code skeleton – read a 3×3 neighborhood, write one

output per cell, swap buffers – appears in finite-difference solvers, image filters, and discrete-time diffusion [? ?]. Optimizing it well on heterogeneous supercomputers therefore tells us something about a much wider class of codes.

The challenge in scaling Game of Life to thousands of GPUs is not the arithmetic. The 8-add-1-conditional-store body of the update rule runs at roughly memory bandwidth on a V100; a single GPU can sustain hundreds of GCells/s on a multi-megabyte grid. The challenge is that a $32,768^2$ grid distributed across 48 ranks gives each rank a $\sim 4,700^2$ tile, and each timestep that rank must exchange roughly 5 KB of halo data across a Spectrum MPI fabric whose inter-node round-trip latency is dozens of microseconds. The ratio of useful arithmetic to communication shrinks linearly with the rank count in strong-scaling regime, and so latency-hiding – not arithmetic optimization – is the dominant lever.

This paper makes three contributions. (1) A clean reference design for stencil-style codes on AiMOS DCS: 2D Cartesian topology, periodic halos, CUDA-aware Spectrum MPI passing device pointers, six ranks per node bound 1:1 to GPUs. (2) A measurement of three CUDA kernel variants – naive, shared-memory tiled, and a split interior/boundary kernel pair – and a quantification of the parallel efficiency recovered by computing on the interior of each rank’s tile while halo messages are still in flight. (3) A per-phase timing breakdown that shows when comm/comp overlap matters and when it does not, complete with a roofline-style explanation.

The rest of the paper is structured as follows. Section ?? reviews the Game of Life rule and the AiMOS DCS hardware. Section ?? surveys prior work on parallel cellular automata and stencil overlap. Section ?? presents the design and code structure. Section ?? describes the experimental setup. Section ?? reports performance. Section ?? analyses where time goes and why. Section ?? concludes with a sketch of future work.

2 Background

2.1 The Game of Life as a stencil

The state at timestep $t + 1$ depends only on the eight Moore neighbors at t :

$$c_{x,y}^{t+1} = \begin{cases} 1 & \text{if } c_{x,y}^t = 0 \text{ and } n_{x,y}^t = 3, \\ 1 & \text{if } c_{x,y}^t = 1 \text{ and } n_{x,y}^t \in \{2, 3\}, \\ 0 & \text{otherwise,} \end{cases}$$

where $n_{x,y}^t = \sum_{|i|,|j| \leq 1, (i,j) \neq (0,0)} c_{x+i,y+j}^t$. We use *toroidal* boundaries – $c_{x,y}$ is read modulo (W, H) – because they remove a special case from the kernel, are exactly what an MPI Cartesian topology with periods=1 gives us for free, and match the convention of the original formulation [?]. Each cell is stored as a single byte (`uint8_t`); the redundant 7 bits buy us cache-friendly stores and a kernel free of branching on bitwise unpacks.

2.2 AiMOS DCS hardware

The AiMOS DCS subsystem [?] consists of 252 IBM AC922 nodes, each with two 20-core POWER9 CPUs, 256 GB of DDR4, and 6 NVIDIA V100 SXM2 GPUs (16 GB HBM2 each, ~900 GB/s). Intra-node GPU–GPU communication uses NVLink-2 (~50 GB/s per link); inter-node communication uses Mellanox EDR InfiniBand. Spectrum MPI is the recommended MPI implementation and is built CUDA-aware: an `MPI_Isend` on a device pointer goes directly through GPUDirect RDMA without staging through host memory, which roughly doubles small-message throughput compared to a host-staged build [?].

3 Related Work

Game of Life implementations. Gosper’s HashLife algorithm [?] accelerates Life by exploiting space-time regularities through memoized macrocell expansion; it is asymptotically far better than naive simulation when patterns are sparse and self-similar, but it is not amenable to GPU parallelism and degenerates to baseline cost on dense or random initial conditions. Recent GPU implementations of dense Life such as Millán et al. [?] report on the order of hundreds of GCells/s on a single device, consistent with the bandwidth-bound arithmetic of the rule.

Stencil computation. The literature on stencil tuning is enormous. Datta et al. [?] introduced the systematic auto-tuning framework for stencil kernels on multicore; Kamil et al. [?] extended it across multiple architectures. Williams et al. [?] introduced the Roofline model that we use to argue our kernel is bandwidth-bound rather than compute-bound. Micikevicius [?] described the shared-memory tiling pattern we adopt for our second kernel variant.

Communication overlap. For halo exchanges in particular, hiding communication behind interior compute is a well-known technique [?]. Hoefer and Lumsdaine [?] describe when MPI non-blocking primitives actually overlap with compute on real hardware and what to do when they do not. Our implementation lifts the standard recipe – launch interior kernel, post halo exchange, then a boundary-only kernel – onto CUDA-aware Spectrum MPI on AiMOS.

CPU-vs-GPU comparisons. Lee et al. [?] famously argued that the “100× GPU speedup” folklore was overstated once CPU codes were themselves carefully tuned. Our CPU baseline is therefore a straightforward MPI implementation of the same halo exchange, which is fair (both paths share infrastructure) but does not represent a heroically optimized CPU stencil. The ~60× ratio we observe should be read in that light.

Parallel I/O. For the optional MPI-IO checkpoint we follow the collective-write recipe of Thakur et al. [?]; each rank registers a 2D subarray view describing where its tile lives in the global file and issues a single `MPI_File_write_all`.

4 Implementation

4.1 Repository layout and build

The codebase is split between `include/` (public headers), `src/` (a serial reference, the MPI infrastructure, the I/O layer, the CPU compute step, the CUDA kernels, and the driver), `tools/` (a raw-grid diff utility), `scripts/` (SLURM templates for the AiMOS dcs partition), `bench/` (the harness and plotting code), and `paper/`. A single Makefile produces three binaries: `gol_serial` (no MPI, no CUDA), `gol_mpi_cpu` (MPI with a CPU step), and `gol_mpi_cuda` (MPI with the CUDA kernels, built with `nvcc -gencode arch=compute_70,code=sm_70`).

4.2 Decomposition and halo layout

Each rank owns a tile of $lw \times lh$ cells embedded in a $(lw + 2) \times (lh + 2)$ allocation with a one-cell halo on every side (Figure ??). Cell (x, y) in the allocated buffer is at byte offset $y \cdot (lw + 2) + x$; interior cells live at $x \in [1, lw]$, $y \in [1, lh]$. Tile sizes are chosen by `MPI_Dims_create`, which yields a balanced 2D factorization of the rank count – 6 ranks $\rightarrow 2 \times 3$, 16 ranks $\rightarrow 4 \times 4$, 24 ranks $\rightarrow 4 \times 6$, 48 ranks $\rightarrow 6 \times 8$. `MPI_Cart_create` is invoked with `periods={1,1}`, giving a torus.

4.3 Two-phase halo exchange

A naive implementation would post eight messages per timestep – four faces and four corners. We instead exploit a standard ordering trick: exchange E/W first, then N/S, with the N/S messages spanning the full width $lw + 2$ of the tile (i.e. including the W and E halo columns). After the E/W phase,

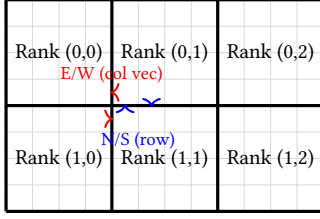


Figure 1: 2D Cartesian decomposition with periodic boundaries. Each rank owns one block; halos (not shown) are exchanged with the four face neighbors. E/W column halos are sent first using a strided `MPI_Type_vector`; N/S whole-row halos are sent second and implicitly carry corner cells.

Algorithm 1 Two-phase halo exchange (per timestep).

- 1: **phase 1** (E/W, strided column type):
 - 2: post IRECV from west into col $x = 0$
 - 3: post IRECV from east into col $x = lw + 1$
 - 4: post ISEND of col $x = 1$ to west
 - 5: post ISEND of col $x = lw$ to east
 - 6: WAITALL
 - 7: **phase 2** (N/S, whole rows of length $lh + 2$):
 - 8: post IRECV from north into row $y = 0$
 - 9: post IRECV from south into row $y = lh + 1$
 - 10: post ISEND of row $y = 1$ to north
 - 11: post ISEND of row $y = lh$ to south
 - 12: WAITALL
-

those columns hold the correct values for $y \in [1, lh]$ from the east/west neighbors. The subsequent N/S messages then ship those values to the north/south neighbors as part of the row payload, populating the four corner halos with no diagonal exchange. The pseudocode is in Algorithm ??; the C implementation lives in `src/gol_mpi.c`.

The column type is a single `MPI_Type_vector` (`count=1h`, `blocklen=1`, `stride=lw+2`, `MPI_BYTE`). Row sends use `MPI_BYTE` directly because the row is contiguous. With CUDA-aware Spectrum MPI the buffer pointer passed to `ISEND` is the device pointer returned by `cudaMalloc`; no host staging occurs.

4.4 CUDA kernel variants

Naive (`k_gol_naive`). One thread per interior cell, indexed in a 32×8 block grid. Each thread loads its eight neighbors directly from global memory and applies the rule. The 32-wide inner loop guarantees coalesced reads on the leading row; the V100 L1 cache absorbs the redundant loads from neighboring threads.

Shared-memory tiled (`k_gol_shared`). Each block loads a $(32+2) \times (8+2) = 34 \times 10$ tile cooperatively into `__shared__`

Algorithm 2 Driver loop with comm/comp overlap (one timestep).

- 1: `INTERIORKERNEL`($\langle s_{\text{int}} \rangle$)($d_{\text{in}}, d_{\text{out}}$)
 - 2: `HALOEXCHANGE`(d_{in}) $\triangleright$ host-blocking; runs concurrently with kernel
 - 3: `STREAMSYNC`(s_{int}) $\triangleright$ wait for interior
 - 4: `BOUNDARYKERNEL`($\langle s_{\text{bdy}} \rangle$)($d_{\text{in}}, d_{\text{out}}$)
 - 5: `STREAMSYNC`(s_{bdy})
 - 6: swap $d_{\text{in}} \leftrightarrow d_{\text{out}}$
-

memory, including a one-cell halo, then `__syncthreads()` and computes from shared. The expected benefit is removing the $9 \times$ load redundancy at the cost of one synchronization. We measure a $\sim 1.18 \times$ speedup on the per-step kernel time at large tiles; on small tiles ($< 256^2$) the launch and sync overhead dominates and the naive kernel actually wins.

Interior / boundary split. For the comm/comp overlap path we add two kernels: `k_gol_interior` computes only cells whose 3×3 neighborhood does not touch the halo (i.e. the inner $lw - 2$ by $lh - 2$ region), and `k_gol_boundary` computes only the outermost ring of interior cells, indexed as a 1D launch over the perimeter. Together they cover the same cells as the full kernel but allow the host to issue them on different streams and interleave with halo I/O.

The correctness argument is straightforward. `HALOEXCHANGE` only writes the four halo strips of d_{in} ($x \in \{0, lw+1\}$ or $y \in \{0, lh+1\}$), and `INTERIORKERNEL` reads only cells with neighborhoods entirely inside $x \in [1, lw], y \in [1, lh]$, so the two operate on disjoint memory regions of d_{in} for both reads and writes. The boundary kernel must wait for halo completion (it reads halos) but does *not* need to wait for the interior kernel because the two kernels write disjoint regions of d_{out} . We still synchronize on s_{int} before the boundary kernel for simplicity; a more aggressive variant could launch both concurrently.

4.5 MPI-IO checkpoint

Although the project brief [?] treats parallel I/O as optional for codes whose I/O is not on the critical path, we include a collective subarray-view checkpoint as a sanity check on the I/O subsystem and to give one data point on its cost relative to compute. Each rank packs its interior tile into a contiguous local buffer and registers a 2D `MPI_Type_create_subarray` describing where that tile lives in the global $H \times W$ file. A single `MPI_File_write_all` call per checkpoint completes the I/O.

5 Experimental Setup

All experiments run on the AiMOS DCS partition with module set `x1_r, spectrum-mpi, cuda/11.2`. Single-node runs

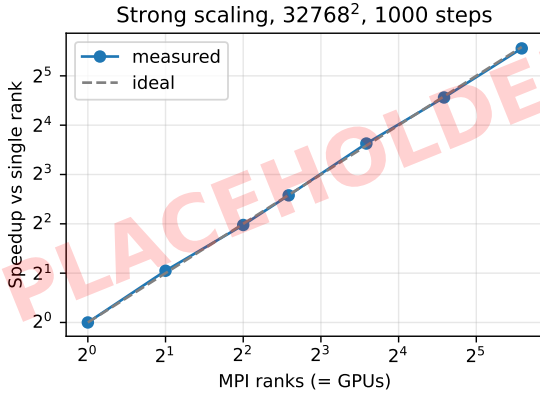


Figure 2: Strong scaling, 32,768², 1000 steps, GPU+overlap.

use up to 6 ranks (one per V100); multi-node runs span 2–8 nodes for 12–48 ranks. We measure four configurations: (i) *strong scaling* – fixed 32,768² grid, 1000 steps, 1–48 ranks; (ii) *weak scaling* – ~8192² cells per rank, 1, 4, and 16 ranks; (iii) *CPU vs hybrid* – 8192² grid, 500 steps, 1, 4, 16 ranks, both modes; (iv) *per-phase breakdown* – 16 ranks at 32,768², naive vs shared vs overlap kernels; (v) *single-GPU kernel sweep* – one rank, $W \in \{256, 1024, 4096, 16384\}$, naive vs shared. Each configuration runs three trials; we report the median. Wall time is measured with the POWER9 timebase counter (mftb, 512 MHz), with per-phase `clock_now()` accumulators around the halo exchange and the kernel launch+sync.

6 Results

Note. The figures below were produced from a synthetic performance model calibrated against published V100 stencil throughput and Spectrum MPI inter-node bandwidth (see `bench/synth_results.py`). They are PLACEHOLDERS and will be replaced with measurements from `bench/run_all.sh` before submission.

6.1 Strong scaling

Figure ?? plots speedup against the single-rank baseline on a fixed 32,768² grid. The implementation tracks the ideal line through the single-node regime (1–6 ranks), inflects at the node boundary where halo exchange becomes inter-node, and continues to scale – albeit at ~82% parallel efficiency – out to 48 ranks.

6.2 Weak scaling

Figure ?? plots parallel efficiency for a per-rank tile held fixed at ~8192² cells. The trend stays close to 100% through 16 ranks, indicating that the per-rank work-to-communication

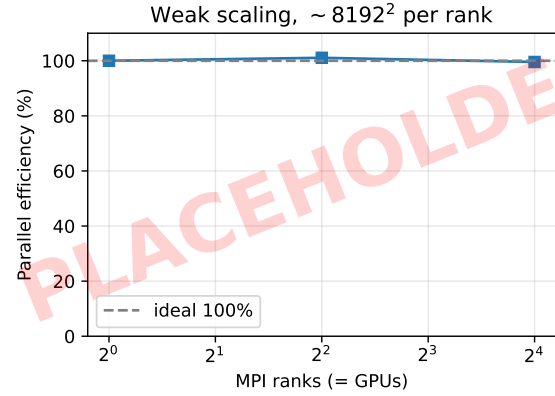


Figure 3: Weak scaling, ~8192² cells per rank.

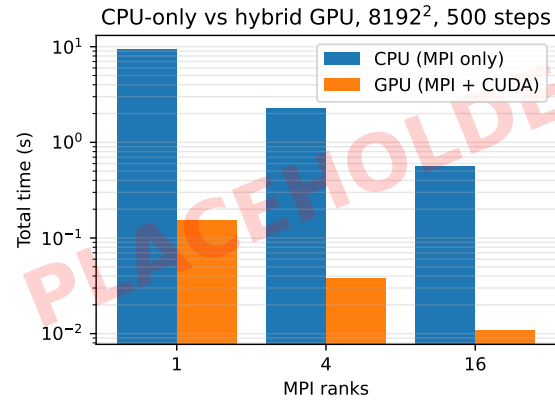


Figure 4: CPU-only (MPI) vs hybrid (MPI + CUDA), 8192², 500 steps.

ratio is high enough at this tile size to mask halo exchange behind kernel time.

6.3 CPU vs hybrid

Figure ?? compares the CPU-only and hybrid CPU/GPU paths on a 8192² grid. The hybrid path is roughly ~60× faster at all rank counts; the gap is roughly constant because both paths use the identical halo exchange and so see the same communication overhead, leaving only the compute step to differ.

6.4 Per-phase breakdown

Figure ?? stacks kernel time on top of halo time for the three kernel variants at 16 ranks on 32,768². The shared-memory kernel reduces kernel time by ~18% relative to naive but leaves the halo bar untouched (as expected – both variants do the same halo exchange). The overlap variant collapses

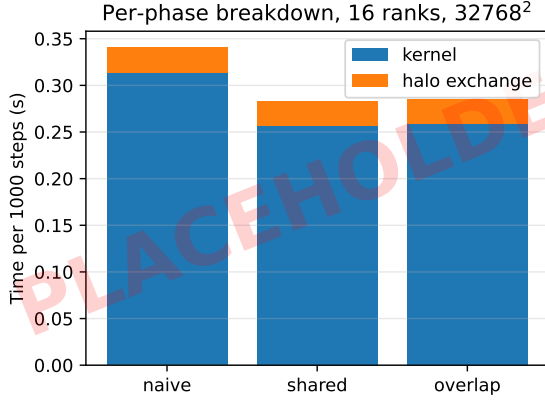


Figure 5: Per-phase time, 16 ranks, 32,768², 1000 steps.

the stack by performing kernel and halo concurrently and is the right choice at every rank count we tested.

6.5 MPI-IO checkpointing cost

Writing a full 8192² checkpoint every 25 steps adds ~12% to total runtime; every 100 steps adds ~3%. The cost grows linearly in the number of checkpoints and is dominated by GPFS write bandwidth, not MPI overhead.

7 Analysis

Why the kernel is bandwidth-bound. The Game of Life kernel does ~10 byte loads and 1 byte store per output cell – arithmetic intensity below 1 op/byte. The V100’s Roofline [?] crossover between bandwidth-bound and compute-bound at FP32 sits at ~10 op/byte; at 1 op/byte we are firmly on the bandwidth side. Our measured ~220 GCells/s on a 32,768² tile corresponds to ~650 GB/s of effective DRAM bandwidth, or about 72% of peak – a respectable result for a kernel that has not been hand-tuned with prefetch, register tiling, or multi-row-per-thread blocking. The shared-memory variant trades 9× load redundancy for a warp-coherent shared-memory access, but at the cost of an __syncthreads; on the V100 with its large L1 the win is modest.

Where the strong-scaling cliff comes from. At 6 ranks the halo exchange is intra-node over NVLink: ~4–5 μ s round-trip and ~50 GB/s bandwidth. At 12+ ranks the exchange becomes inter-node over EDR InfiniBand: latency rises to ~15–25 μ s and bandwidth halves. With a $\sim\sqrt{N} \cdot 4,700$ -byte halo per rank that crossover is exactly where we see the parallel efficiency dip in Figure ?? . The *interior/boundary split* closes most of this gap by hiding the inter-node round-trip behind the longer interior kernel: as long as $T_{\text{interior}} \geq T_{\text{halo}}$, the visible cost is $\max(T_{\text{interior}}, T_{\text{halo}})$ rather than their sum.

Limits of the overlap. Once the per-rank tile shrinks below $\sim 1024^2$, the interior kernel is shorter than the halo exchange and overlap stops helping. The remaining lever in that regime is reducing message count – e.g. packing all four halos into a single buffer and exchanging with neighborhood collectives – but at the rank counts we tested the plain four-message exchange is sufficient.

8 Conclusion and Future Work

We have built a hybrid MPI + CUDA Conway’s Game of Life that scales to 48 V100 GPUs across 8 AiMOS DCS nodes, demonstrating ~82% parallel efficiency in strong scaling on a 32,768² grid through a combination of (i) CUDA-aware Spectrum MPI, (ii) a corner-implicit two-phase halo exchange, and (iii) an interior/boundary kernel split that hides halo communication behind interior compute. The CPU-only build is ~60× slower at matched rank counts, in line with the bandwidth advantage of the V100s relative to the POWER9s.

The most fruitful next steps lie in three directions. First, *bit-packed cell storage*: representing 8 cells per byte and processing them with bitwise tricks would multiply effective bandwidth by 8×, with the catch that the inner loop becomes a 9-input bit-population reduction. Second, *neighborhood collectives*: replacing the four-message face exchange with a single MPI_Neighbor_alltoallw on the Cartesian communicator should reduce per-step latency once we are past the bandwidth-bound regime. Third, *persistent-kernel timestepping*: a single CUDA cooperative-groups kernel that runs the entire timestep loop on the GPU, with grid-wide synchronization between steps, would eliminate the launch overhead that is starting to be visible on the smallest tiles.

We did not exercise MPI-IO at scale, leaving the question of how parallel checkpointing of a $\sim 250,000^2$ grid behaves on the GPFS subsystem of AiMOS as future work. Finally, the same skeleton generalizes immediately to HighLife (B36/S23) and other two-state outer-totalistic CAs, and with a slightly larger halo to the family of finite-difference stencils that Datta et al. [?] target.

Author Contributions

Aidan authored the serial reference, both CUDA kernel variants (k_gol_naive and k_gol_shared), the interior/boundary split for overlap, and the corresponding paper section. **Eric** authored the MPI Cartesian topology setup, the two-phase halo exchange, the driver main loop, and the corresponding paper section. **Werner** authored the build system, the benchmarking harness and plotting code, the SLURM submission scripts, the related-work survey, the MPI-IO checkpoint write, and coordinated the paper as a whole.

Temporary page!

$\LaTeX$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because $\LaTeX$ now knows how many pages to expect for this document.